Министерство образования и науки Российской Федерации

Уральский федеральный университет
имени первого Президента России Б.Н.Ельцина

Е.И. Кац, А.М. Китаев

ГЕОМЕТРИЧЕСКОЕ МОДЕЛИРОВАНИЕ

Электроный текстовый ресурс

Методические указания к лабораторным работам по курсу «Геометрическое моделирование» для студентов дневной формы обучения направления 09.03.01 «Информатика и вычислительная техника».

Подготовлено кафедрой «Информационные технологии и автоматизация проектирования»

Определены формулировки заданий и описаны подходы к их решению с обоснованием их рациональности. Приведены некоторые неявные вопросы.

Екатеринбург

2017

ОГЛАВЛЕНИЕ

Оглавление **2**

1. Лабораторная работа № 1 **3**

1.1. Постановка задачи 3

1.2. Проектирование программы 3

1.2.1. Формат файла сцены 3

1.2.2. Считывание из файла 4

1.2.3. Структура данных программы 5

1.3. Неочевидные вопросы 10

2. Лабораторная работа № 2 **11**

2.1. Постановка задачи 11

2.2. Проектирование программы 11

2.2.1. Формат файла сцены 11

2.2.2. Визуализация сегментов кривых 12

2.3. Возможные вопросы 14

2.4. Реализация с помощью OpenGL 14

3. Лабораторная работа № 3 **16**

3.1. Постановка задачи 16

3.2. Проектирование программы 16

3.2.1. Формат файла сцены 16

3.2.2. Визуализация порций поверхности 18

3.3. Возможные вопросы 18

3.4. Реализация с помощью OpenGL 18

ГЛАВА 1. ЛАБОРАТОРНАЯ РАБОТА № 1

1.1. Постановка задачи

Написать программу, которая:

– загружает из текстового файла трёхмерную сцену, состоящую из отрезков, описанных в однородной системе координат $[X, Y, Z, H]$;
– отображает трёхмерную сцену (проекцию сцены на плоскость xy декартовой системы координат, ось x направлена вправо, y – вверх, z – на наблюдателя, начало координат расположено в центре области отображения);
– осуществляет над сценой аффинные преобразования: параллельный перенос, масштабирование, поворот вокруг осей координат и косой сдвиг;
– осуществляет над сценой ОПП[1] с заданным параметром;
– параметры преобразований задаются пользователем;
– осуществляет вписывание сцены в область отображения без искажений (одинаковое масштабирование по всем осям, отсутствие вращений).

1.2. Проектирование программы

1.2.1. Формат файла сцены

Рассмотрим задачу описания трёхмерной сцены в текстовом файле. Так как сцена состоит из отрезков в однородной системе координат, можно, к примеру, задавать каждый отрезок восемью координатами, по четыре координаты для каждого конца, $X_1\ Y_1\ Z_1\ H_1\ X_2\ Y_2\ Z_2\ H_2$, разделённые пробелом. Каждый отрезок может быть записан в своей строке. Тогда содержание файла будет примерно таким:

```
0.0 0.0 0.0 1.0 10.0 12.0 15.0 1.0
10.0 12.0 15.0 1.0 10.0 12.0 0.0 1.0
```

[1] ОПП – одноточечное проективное преобразование

Ещё один способ задать сцену из отрезков – отделить описание точек в от топологии (связей между ними). В таком случае, содержание файла для сцены из первого примера будет выглядеть так:

```
0.0 0.0 0.0 1.0
10.0 12.0 15.0 1.0
10.0 12.0 0.0 1.0
0 1
1 2
```

В начале файла описаны точки в каждой строке по четыре координаты. Координаты разделены пробелами. Далее в каждой строке по одному записаны отрезки. Каждый отрезок описывается двумя порядковыми номерами точек, соответствующих концам отрезка. Такой вариант описания сцены является более удобным, так как в геометрических фигурах каждая вершина часто является концом нескольких отрезков.

1.2.2. Считывание из файла

Для упрощения считывания из файла трёхмерной сцены можно ввести некоторые изменения в ранее предложенные форматы файла, а именно – ввести «блоки» точек и топологии. Содержание файла с блоками может выглядеть так:

```
3
0.0 0.0 0.0 1.0
10.0 12.0 15.0 1.0
10.0 12.0 0.0 1.0
2
0 1
1 2
```

Как видно из приведённого примера, к предыдущему описанию добавлено две строки. В первой указано количество точек, записанных в файле, во второй – количество отрезков. Такой подход позволяет не тратить время на анализ инфор-

мации в строке, а заранее предполагать что именно, точка или отрезок, описаны в строке. Программа может прочитать из первой строки количество точек, затем повторить считывание точек из последующих строк указанное количество раз. Аналогично программа поступит и с «блоком» отрезков. Конечно, при таком подходе нужно условиться, что количества точек и отрезков всегда указаны верно, в противном случае программа не сможет прочитать файл сцены.

1.2.3. Структура данных программы

Необходимо определиться со структурой данных, которая будет хранить сцену. В зависимости от порядка обработки данных структура хранимых переменных может меняться.

Самый примитивный класс, с которым необходимо работать, – это класс точки. Лучше всего хранить точку в виде массива из четырёх чисел с плавающей запятой. Таким образом, можно хранить однородные координаты точки и иметь доступ к каждой из координат по её номеру в массиве.

$$\begin{bmatrix} X & Y & Z & H \end{bmatrix}$$

Данные о геометрии и топологии предлагается хранить таким же образом, как и в файле: точки хранить в отдельном массиве, для каждого отрезка хранить номера его концов в массиве точек.

Разделять информацию о точках с информацией об отрезках более разумно, так как: работа с геометрической информацией отделена от работы с топологической (алгоритмы преобразования работают с координатами точек), объём используемой памяти обычно меньше (так как устраняется дублирование геометрической информации о точках при описании геометрических фигур).

Матрицы преобразования. Матрицы преобразования отличаются для точек, представленных в виде матрицы строки и матрицы столбца. Условимся, что точки представлены в виде матрицы строки. Тогда матрицы преобразования

будут выглядеть так[2]:

$$\begin{bmatrix} a & b & c & p \\ d & e & f & q \\ h & i & j & r \\ l & m & n & s \end{bmatrix},$$

где a, e, j отвечают за масштабирование соответственно по осям x, y, z; l, m, n отвечают за параллельный перенос соответственно по осям x, y, z

Матрица параллельного переноса на вектор $\vec{a}\,(x, y, z)$:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ x & y & z & 1 \end{bmatrix}.$$

Матрица масштабирования по оси X:

$$\begin{bmatrix} k_x & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Матрица масштабирования по оси Y:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & k_y & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

[2]См. Роджерс Д. «Математические основы машинной графики»

Матрица масштабирования по оси Z:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & k_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Матрица поворота вокруг оси X на угол α:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\alpha & \sin\alpha & 0 \\ 0 & -\sin\alpha & \cos\alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Матрица поворота вокруг оси Y на угол α:

$$\begin{bmatrix} \cos\alpha & 0 & -\sin\alpha & 0 \\ 0 & 1 & 0 & 0 \\ \sin\alpha & 1 & \cos\alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Матрица поворота вокруг оси Z на угол α:

$$\begin{bmatrix} \cos\alpha & \sin\alpha & 0 & 0 \\ -\sin\alpha & \cos\alpha & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Матрица косого сдвига оси X по оси Y с коэффициентом k:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ k & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Матрица косого сдвига оси X по оси Z с коэффициентом k:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ k & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Матрица косого сдвига оси Y по оси X с коэффициентом k:

$$\begin{bmatrix} 1 & k & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Матрица косого сдвига оси Y по оси Z с коэффициентом k:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & k & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Матрица косого сдвига оси Z по оси X с коэффициентом k:

$$\begin{bmatrix} 1 & 0 & k & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Матрица косого сдвига оси Z по оси Y с коэффициентом k:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & k & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Матрица ОПП по оси X с фокусным расстоянием f_x:

$$\begin{bmatrix} 1 & 0 & 0 & \frac{1}{f_x} \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Матрица ОПП по оси Y с фокусным расстоянием f_y:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & \frac{1}{f_y} \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Матрица ОПП по оси Z с фокусным расстоянием f_z:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & \frac{1}{f_z} \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Порядок обработки. Существует два варианта порядка обработки точек:

1. Хранение исходной сцены, обрабатываемой через итоговое преобразование перед отрисовкой, итоговое преобразование – это матрица, являющаяся результатом произведения матриц всех преобразований в соответствующем порядке.
2. Хранение преобразованной сцены с последовательным применением преобразований.

1.3. Неочевидные вопросы

1. Для ОПП можно задавать фокусное расстояние или непосредственно величину соответствующего элемента матрицы преобразования. Что именно используется в программе – часто задаваемый вопрос.
2. **Сколько существует разных простых косых сдвигов?** Существует шесть косых сдвигов: x/y (x по y) – элемент d матрицы преобразования, x/z – элемент h, y/x, y/z, z/x, z/y

ГЛАВА 2. ЛАБОРАТОРНАЯ РАБОТА № 2

2.1. Постановка задачи

Дополнить программу первой лабораторной работы возможностью задания в исходном файле сегментов рациональной кубической кривой Безье, заданной в базисе Болла. Обеспечить выполнение над этими сегментами всех аффинных и проективных преобразований, реализованных ранее, и визуализацию ортогональной проекции сегментов на плоскость xy.

2.2. Проектирование программы

2.2.1. Формат файла сцены

В первой работе мы выбрали способ представления сцены, составленной из отрезков. Сейчас нам нужно расширить его для представления сегментов рациональных кубических кривых. Для однозначного задания одного такого сегмента достаточно задать четыре управляющие точки. Продолжая использовать идею о разделении геометрической и топологической информации, логично сохранить неизменным способ представления набора точек, а сегмент кривой задавать, указывая четыре номера, которые будут номерами его управляющих точек в наборе точек. При таком подходе, каждому сегменту кривой мы поставим в соответствие одну строку в исходном файле, содержащую четыре целых числа. Дуга окружности радиуса 1 с центром в начале координат, лежащая в первом квадранте $X \geq 0, Y \geq 0$, может тогда быть представлена, как:

```
3
1.0 0.0 0.0 1.0
0.707 0.707 0.0 0.0707
0.0 1.0 0.0 1.0
1
0 1 1 2
```

или

```
4
1.0 0.0 0.0 1.0
0.707 0.707 0.0 0.0707
0.0 1.0 0.0 1.0
0.707 0.707 0.0 0.0707
1
0 1 3 2
```

При таком подходе все преобразования, реализованные в задаче №1, продолжают работать без каких-либо изменений. Для того чтобы в этом убедиться, достаточно рассмотреть применение произвольного преобразования, заданного матрицей T, к произвольному сегменту $P(t)$.

Результатом применения преобразования будет сегмент $Q(t) = P(t) \cdot T$. Подставим выражение для $P(t) = \sum\limits_{i=0}^{3} P_i \cdot g_i(t)$, где
$g_i(t)$ – базисные функции, а
P_i – управляющие точки.
Получим:
$Q(t) = (\sum\limits_{i=0}^{3} P_i \cdot g_i(t)) \cdot T = \sum\limits_{i=0}^{3} (P_i \cdot T) \cdot g_i(t) = \sum\limits_{i=0}^{3} Q_i \cdot g_i(t)$,
здесь $Q_i = P_i \cdot T$ - некоторые новые точки. Таким образом мы показали, что сегмент $Q(t)$ является сегментом рациональной кубической кривой и его управляющие точки можно получить, применяя преобразование T к управляющим точкам Pi исходного сегмента $P(t)$.

2.2.2. Визуализация сегментов кривых

Прежде всего необходимо вспомнить, как выглядит базис Болла.

$g_0(t) = (1-t)^2$,

$g_1(t) = 2 \cdot t \cdot (1-t)^2$,

$g_2(t) = 2 \cdot (1-t) \cdot t^2$,

$g_3(t) = t^2$.

Самый простой подход к визуализации любой кривой, заданной параметрическим уравнением, заключается в вычислении какого-то количества точек на этой кривой и визуализации самих этих точек или отрезков их соединяющих. В первом варианте не очень просто обеспечить непрерывность изображаемого сегмента, поэтому чаще используют второй. В этом случае мы выполняем аппроксимацию исходного сегмента ломаной. Возникает естественный вопрос о точности такой аппроксимации. Было бы логично поставить задачу об аппроксимации с заданной пользователем точностью. Кроме того, достаточно естественно было бы дополнить ее требованием минимизации количества отрезков в аппроксимирующей ломаной. Такая задача оказалась бы совсем нетривиальной. К счастью, в постановке задачи нет никаких требований по точности аппроксимации, следовательно мы можем применить любой подход, который покажется нам достаточно простым и достаточно разумным.

Проще всего вычислять точки на кривой, изменяя значение параметра от 0 до 1 с постоянным шагом. Тогда k-я вершина аппроксимирующей ломаной будет вычисляться , как:

$R_k = \sum\limits_{i=0}^{3} P_i \cdot g_i(t_k), k = \overline{0, N}$, где

$t_k = k/N$, а N – количество отрезков ломаной.

Количество точек, очевидно, будет на единицу больше количества отрезков. Можно вычислить все точки по этой формуле, сохранить их в массиве и потом нарисовать отрезки, соединяющие пары последовательных точек. Более разумным кажется подход, при котором мы запоминаем только одну точку и рисуем отрезок соединяющий предыдущую вычисленную точку с текущей. В рамках этой лабораторной работы нужно, наверное, так и поступить. Если же говорить о визуализации сложных сцен, содержащих большое количество кривых, то на практике чаще используют первый подход. Связано это с тем, что передавать на отображение в видеокарту массив координат точек за один раз оказывается более эффективно, чем передавать их по одной. Под эффективностью, в такого рода задачах, чаще всего понимают быстродействие.

Окончательно получается, что для визуализации одного сегмента нам нужно включить в программу всего один цикл, в котором вычисляется очередная точка на кривой и отображается отрезок, соединяющий эту точку с предыдущей.

2.3. Возможные вопросы

1. Что произойдет с изображением, если все координаты одной из управляющих точек умножить на одно и то же число?
2. Как нужно изменить координаты управляющих точек, чтобы получить параболу (гиперболу, эллипс)?

2.4. Реализация с помощью OpenGL

Вместо того чтобы самостоятельно программировать расчет точек на кривой и генерацию соединяющих их отрезков, можно воспользоваться функционалом одной из библиотек компьютерной графики. Например OpenGL. Визуализацию рациональных параметрических кривых в OpenGL осуществляют методы:

```
gluBeginCurve
gluEndCurve
gluNurbsCurve
```

Важно отдавать себе отчет в том, что OpenGL использует базис Бернштейна:

$b_0(t) = (1-t)^3,$

$b_1(t) = 3 \cdot t \cdot (1-t)^2,$

$b_2(t) = 3 \cdot (1-t) \cdot t^2,$

$b_3(t) = t^3.$

Нам же необходимо выполнить визуализацию в базисе Болла, следовательно придется самостоятельно пересчитать заданные пользователем управляющие точки. Для этого нужно найти новые управляющие точки Q_i, которые определят в базисе Бернштейна тот же сегмент кривой, который в базисе Болла

определяется заданными точками P_i. На формальном языке это означает, что нужно найти 4 точки Q_i, удовлетворяющие тождеству:

$\sum\limits_{i=0}^{3} P_i \cdot g_i(t) \equiv \sum\limits_{i=0}^{3} Q_i \cdot b_i(t)$, где

$g_i(t)$ - базис Болла,

$b_i(t)$ - базис Бернштейна,

P_i - заданные управляющие точки.

ГЛАВА 3. ЛАБОРАТОРНАЯ РАБОТА № 3

3.1. Постановка задачи

Дополнить программу второй лабораторной работы возможностью задания в исходном файле порций рациональной кубической поверхности Безье, заданной в базисе Болла. Обеспечить выполнение над этими порциями всех аффинных и проективных преобразований, реализованных ранее, и визуализацию ортогональной проекции порций на плоскость xy.

3.2. Проектирование программы

3.2.1. Формат файла сцены

Порция рациональной кубической поверхности однозначно задается 16 ю своими управляющими точками P_{ij}, $i = \overline{0,3}$; $j = \overline{0,3}$. По аналогии со второй задачей, логично задавать одну порцию указанием 16-ти целых чисел, являющихся номерами ее управляющих точек в общем наборе точек. Для примера рассмотрим представление части тора, являющейся результатом поворота дуги окружности, лежащей в плоскости xz, на $90°$ градусов вокруг оси z. Она может быть представлена так:

```
16
2.0 0.0 0.0 1.0
1.141 0.0 0.707 0.707
1.141 0.0 0.707 0.707
1.0 0.0 1.0 1.0
1.141 1.141 0.0.707
1.0 1.0 0.5 0.5
1.0 1.0 0.5 0.5
0.707 0.707 0.707 0.707
1.141 1.141 0.0.707
```

```
1.0 1.0 0.5 0.5
1.0 1.0 0.5 0.5
0.707 0.707 0.707 0.707
0.0 2.0 0.0 1.0
0.0 1.141 0.707 0.707
0.0 1.141 0.707 0.707
0.0 1.0 1.0 1.0
1
0 1 2 3
4 5 6 7
8 9 10 11
12 13 14 15
```

Можно и существенно более компактно:

```
9
2.0 0.0 0.0 1.0
1.141 0.0 0.707 0.707
1.0 0.0 1.0 1.0
0.0 2.0 0.0 1.0
0.0 1.141 0.707 0.707
0.0 1.0 1.0 1.0
1.141 1.141 0.0.707
1.0 1.0 0.5 0.5
0.707 0.707 0.707 0.707
1
0 1 1 2
6 7 7 8
6 7 7 8
3 4 4 5
```

Если принять такой подход, то выполнение преобразований над порциями снова не потребует каких бы то ни было затрат. Доказательство этого факта полностью аналогично доказательству, приведенному выше для сегментов кривых.

3.2.2. Визуализация порций поверхности

Уравнение порции рациональной кубической поверхности Безье имеет вид:

$$P(u,v) = \sum_{i=0}^{3}\sum_{j=0}^{3} P_{ij} \cdot g_i(u) \cdot g_j(v), 0 \leq (u,v) \leq 1$$

Для визуализации нам снова, очевидно, потребуется вычислить какое-то количество точек на поверхности с последующей аппроксимацией поверхности набором отрезков. В качестве самого простого способа можно выбрать отображение прямоугольной сетки с узлами, расположенными в вычисленных точках. Точки опять проще всего вычислять для равноотстоящих значений параметров.

$R_{ks} = P(u_k, v_s)$, где

$u_k = k/N$,

$v_s = s/N$,

N – количество ячеек сетки по одному параметру.

Для построения сетки теперь нужно просто соединить полученные точки отрезками.

3.3. Возможные вопросы

1. Что произойдет с изображением, если все координаты одной из управляющих точек умножить на одно и то же число?
2. Что нужно изменить в исходном файле, чтобы изменилось положение одного угла порции поверхности.

3.4. Реализация с помощью OpenGL

Вместо того чтобы самостоятельно программировать расчет точек на поверхности и генерацию соединяющих их отрезков, можно воспользоваться функ-

ционалом одной из библиотек компьютерной графики. Например OpenGL. Визуализацию рациональных параметрических поверхностей в OpenGL осуществляют методы:

```
gluBeginSurface
gluEndSurface
gluNurbsSurface
```

Так же как и во второй задаче, здесь будет необходимо предварительно пересчитать управляющие точки.

Учебный электронный текстовый ресурс

Кац Евгений Исакович
Китаев Андрей Михайлович

ГЕОМЕТРИЧЕСКОЕ МОДЕЛИРОВАНИЕ
МЕТОДИЧЕСКИЕ УКАЗАНИЯ К ЛАБОРАТОРНЫМ РАБОТАМ

Редактор *А.В. Овчинникова*
Компьютерная вёрстка *А.М. Китаев*

Рекомендовано Методическим советом
Разрешен к публикации 13.11.2017
Электронный формат – pdf
Объём 0,81 уч.-изд.л.

Уральский федеральный университет имени первого Президента России Б.Н. Ельцина

620002, Екатеринбург, ул. Мира, 19

Информационный портал УрФУ
http://study.urfu.ru